PROGRAMACIÓN DE SERVICIOS Y PROCESOS

Unidad 1. Programación multiproceso

Desarrollado por Adrián Velasco Mañas y Daniel Montañez Pozo

Del proceso a su gestión en Java.

01 **Fundamentos**
Programa, proceso y servicio

02 **Planificación y multitarea**
Planificador, tipos de sistemas y reparto de la CPU

03 **Concurrencia**
Hilos frente a procesos; programación paralela y distribuida

04 **Ciclo de vida y cooperación**
Estados, comunicación (IPC) y sincronización

05 **Multiproceso en Java**
Process, ProcessBuilder, flujos de E/S y casos prácticos

Tres conceptos que no se confunden.

Programa

EL OBJETO ESTÁTICO

- Conjunto de instrucciones ejecutables directamente por una máquina.
- Se almacena en un fichero binario, en un medio secundario.

Proceso

LA ENTIDAD DINÁMICA

- Instancia de un programa en ejecución.
- Usa memoria, procesador y dispositivos de E/S.

Servicio

ATIENDE A OTROS

- Proceso que presta servicio a otros, del mismo equipo o en red.
- Corre en segundo plano, sin usuario; lo inicia el SO en el arranque.

Ejecutar un programa = crear un proceso. Un servicio es un proceso que permanece activo para atender a otros.

Anatomía de un proceso.

El número de procesos simultáneos supera ampliamente el de núcleos. Al **sistema operativo** le toca repartir los recursos de forma ordenada, justa y eficiente.

LINUX	ps · ps -f · ps -Af
MACOS	ps
WINDOWS	tasklist

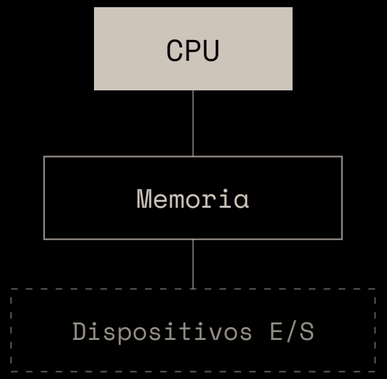


El planificador reparte la CPU.

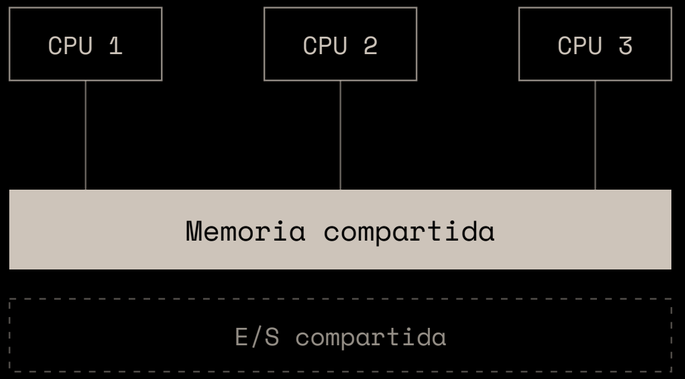
- **Maximizar** el rendimiento del sistema.
- **Maximizar** la equidad en el reparto.
- **Minimizar** los tiempos de espera.
- **Minimizar** los tiempos de respuesta.

LARGO PLAZO	Planificador de trabajos: decide qué procesos se admiten.
MEDIO PLAZO	Gestión de memoria: suspensión y reanudación de procesos.
CORTO PLAZO	Planificador del procesador: reparte el tiempo de CPU.

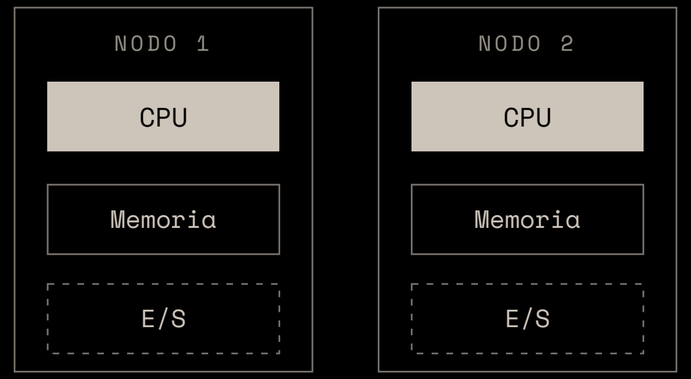
Según el número de procesadores.



Multiprogramación: varios procesos ejecutándose de forma concurrente.



Memoria y E/S compartidas entre todos los procesadores.



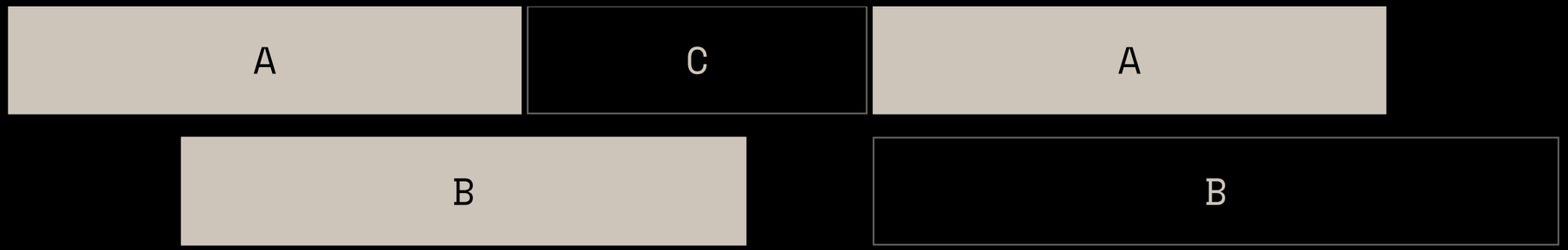
Cada procesador posee su propia memoria y dispositivos de E/S.

Compartir el procesador a lo largo del tiempo.

UN ÚNICO NÚCLEO · TIEMPO COMPARTIDO



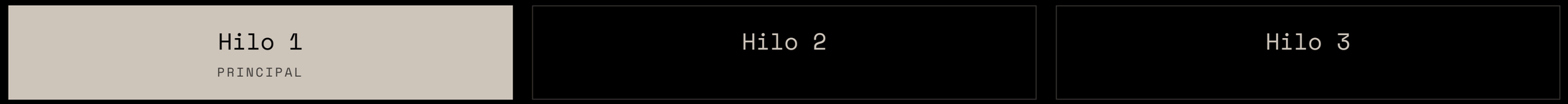
DOS NÚCLEOS · PARALELISMO REAL



Ilusión de simultaneidad: la CPU alterna los procesos en intervalos muy cortos.

Aprovechar la espera: mientras un proceso espera E/S, la CPU ejecuta otro.

El hilo, la unidad mínima de procesamiento.



PROCESOS

- Tienen uno o más hilos, e independientes entre sí.
- Los gestiona el sistema operativo.
- Se comunican a través del SO.

HILOS

- Existen siempre dentro de un proceso.
- Comparten los recursos del proceso directamente.
- Los gestiona el propio proceso, que controla su comunicación.

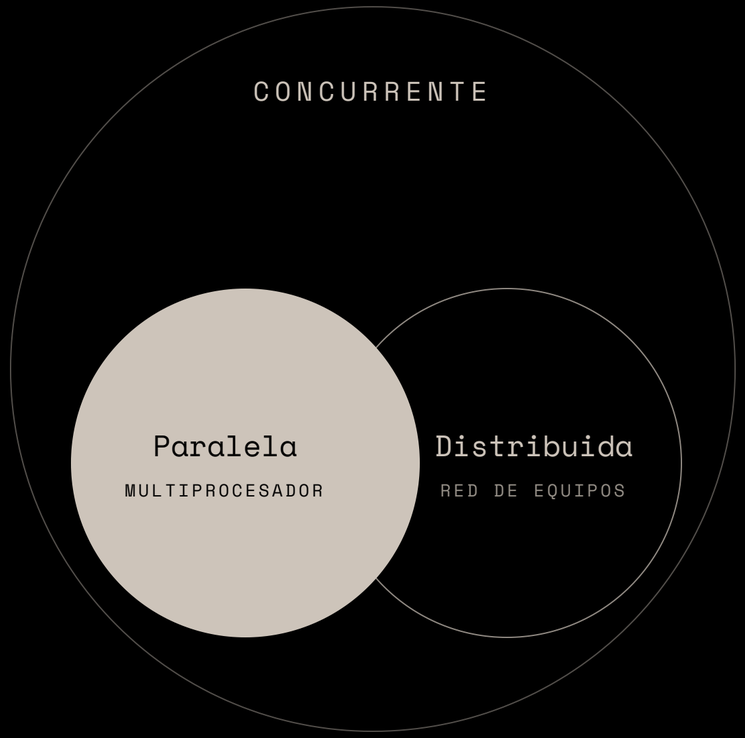
Concurrencia: ventajas y coste.

Procesos que se ejecutan simultáneamente en un intervalo de tiempo: simultaneidad **real** (multiprocesador) o **simulada** (un procesador). Es también el conjunto de técnicas para que varios procesos cooperen.

- **Más rendimiento:** la CPU pasa a otro proceso si el actual espera E/S.
- **Menor tiempo de respuesta:** el tiempo se reparte entre todos.

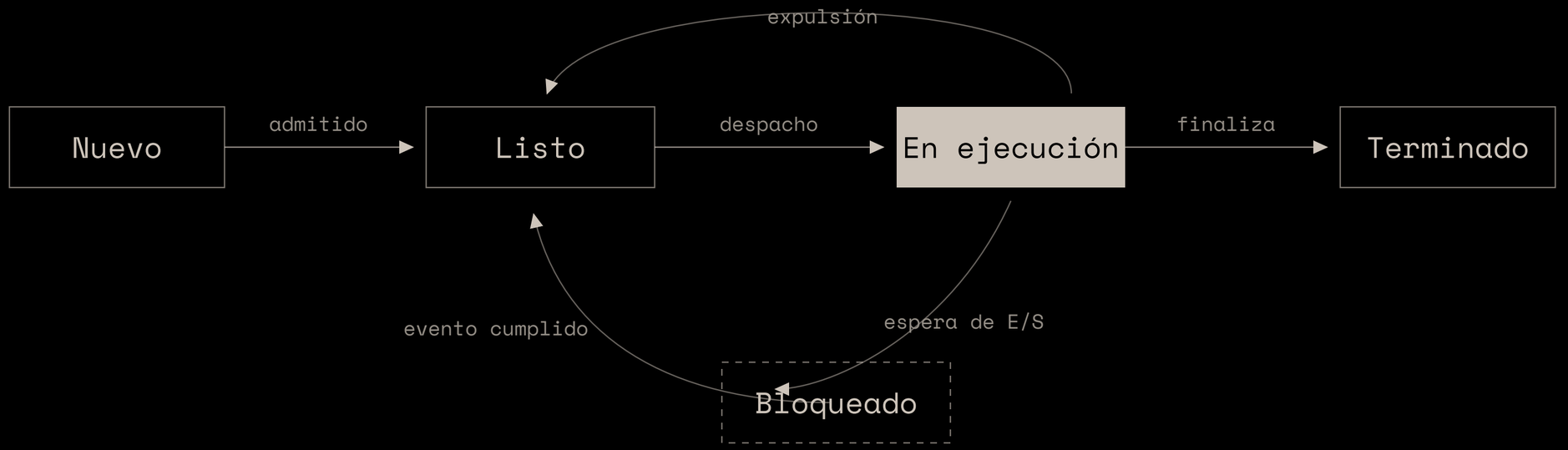
- Sincronización y comunicación más difíciles de implementar.
- La sobrecarga puede anular la mejora de rendimiento.
- En sistemas distribuidos, la red puede ser cuello de botella.

Tres formas de ejecutar varios procesos a la vez.



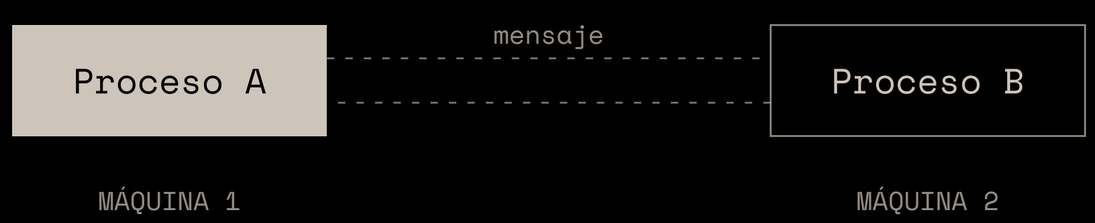
- **Concurrente.** Simultaneidad real o simulada, en uno o varios procesadores.
- **Paralela.** Ejecución en varios procesadores de un mismo equipo.
- **Distribuida.** Procesos repartidos por una red de equipos.

Del nacimiento a la terminación.

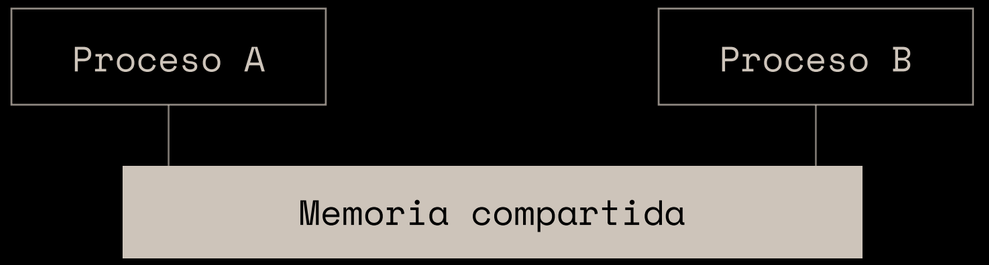


- **Nuevo:** recién creado, espera ser admitido.
- **Listo:** en memoria, esperando CPU.
- **En ejecución:** usa la CPU; uno por núcleo.
- **Bloqueado:** espera un evento (E/S) · **Terminado:** libera recursos.

Dos maneras de comunicarse.



Forma habitual con procesos en máquinas distintas; viaja según un protocolo previo.



Solo en la misma máquina; sincroniza según el valor o estado del recurso.

ALTERNATIVAS · SOCKETS · FLUJOS DE E/S · RPC · PERSISTENCIA · SERVICIOS

FLUJOS DE E/S: LA QUE USAN LOS EJEMPLOS DE ESTA UNIDAD

Cuatro mecanismos y su equivalente en Java.

EJECUCIÓN	`ProcessBuilder.start()`
ESPERA	`Process.waitFor()`
GENERAR CÓDIGO	`System.exit(código)`
OBTENER CÓDIGO	`Process.exitValue()`

Por convención, el código **0** indica terminación correcta.



Dos formas de crear procesos, una clase para controlarlos.

Runtime

Interactúa con el JRE: lanzar procesos, invocar al recolector de basura, consultar la memoria libre. Una única instancia por aplicación.

```
Runtime.getRuntime()
```

ProcessBuilder

Gestiona los atributos de un proceso (comando, directorio, entorno, redirecciones) y lo lanza.

```
pb.start()  →  Process
```

Process y ProcessBuilder.

`destroy()`	Destruye el proceso
`exitValue()`	Valor de retorno al finalizar
`getErrorStream()`	InputStream de error
`getInputStream()`	InputStream de salida normal
`getOutputStream()`	OutputStream de entrada
`isAlive()`	Sigue en ejecución
`waitFor()`	Espera a que termine

`start()`	Inicia el proceso
`command()`	Programa y argumentos
`directory()`	Directorio de trabajo
`environment()`	Entorno de ejecución
`redirectError()`	Destino de errores
`redirectInput()`	Origen de entrada
`redirectOutput()`	Destino de salida

Lanzar un proceso con ProcessBuilder.

```
// Windows
ProcessBuilder pb = new ProcessBuilder("cmd", "/C", "dir");
// Linux
ProcessBuilder pb = new ProcessBuilder("/bin/bash", "-c", "ls");
// macOS
ProcessBuilder pb = new ProcessBuilder("sh", "-c", "ls");

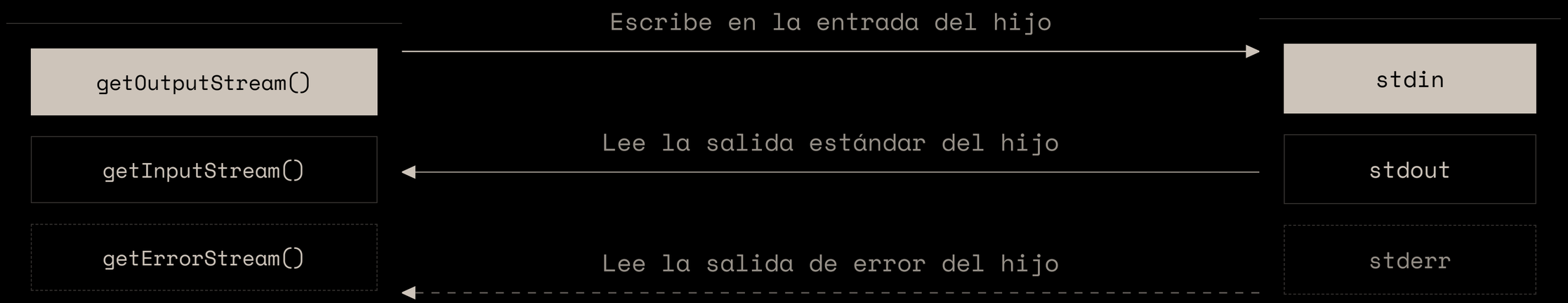
Process p = pb.start();
```

PROCESSBUILDER(LIST<STRING> COMMAND) · PROCESSBUILDER(STRING... COMMAND)

CUATRO PASOS

- **Construir** — comando y argumentos.
- **Configurar** — directorio, entorno, redirecciones.
- **Iniciar** — start() lanza el proceso.
- **Controlar** — con el objeto Process.

Un subproceso no tiene consola. El padre accede a sus flujos.



redirectErrorStream(true) junta el error con la salida estándar. Los flujos se envuelven con **BufferedReader** y **PrintWriter**.

Decoradores sobre los flujos del proceso.

LECTURA

```
Process p = pbuilder.start();
BufferedReader out = new BufferedReader(
  new InputStreamReader(
    p.getInputStream()));
String linea;
while ((linea = out.readLine()) != null) {
  System.out.println("> " + linea);
}
out.close();
```

readLine() devuelve **null** al no quedar líneas.

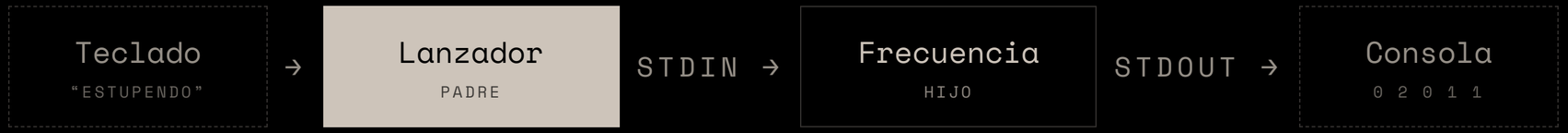
ESCRITURA

```
PrintWriter toProcess = new PrintWriter(
  new BufferedWriter(
    new OutputStreamWriter(
      p.getOutputStream(), "UTF-8")),
  true);

toProcess.println("Enviar al hijo");
```

Con **autoflush**, cada línea llega al hijo al instante.

Lanzador ejecuta Frecuencia y le envía un texto.



HIJO · FRECUENCIA.JAVA

```
Scanner sc = new Scanner(System.in);
String input = sc.nextLine();
for (char c : input.toLowerCase().toCharArray()) {
  switch (c) {
    case 'a': countA++; break;
    case 'e': countE++; break;
    // ... i, o, u
  }
}
System.out.println(countA + " " + countE + ...);
```

PADRE · LANZADOR.JAVA

```
String cp = System.getProperty("java.class.path");
ProcessBuilder pb =
  new ProcessBuilder("java", "-cp", cp, "Frecuencia");
Process process = pb.start();

writer.write(inputString);
writer.newLine();
writer.flush();  writer.close();

String line = reader.readLine();
process.waitFor();
```

newLine() y **close()** marcan el fin de la entrada: sin ellos, Frecuencia esperaría datos eternamente.

Se configura en ProcessBuilder antes de iniciar.

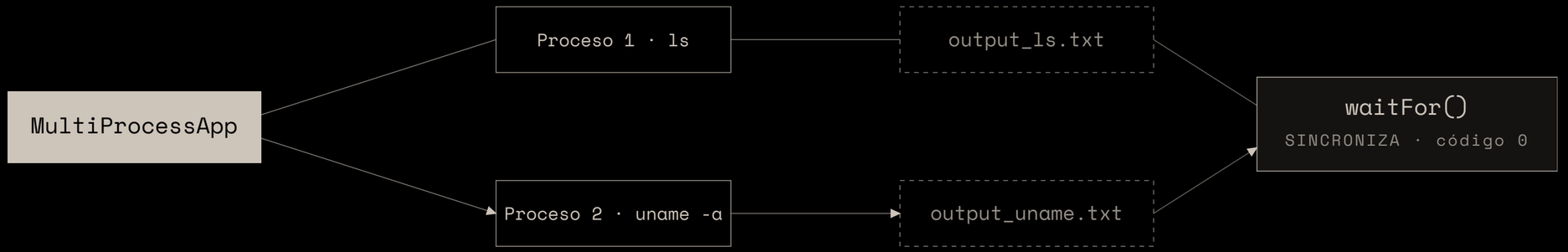
`redirectOutput(File)`	`stdout`
`redirectInput(File)`	`stdin`
`redirectError(File)`	`stderr`

Vía habitual para guardar resultados en ficheros de log o de errores.

```
pb.redirectOutput(new File("salida.txt"));
```

`DISCARD`	Se descarta
`to(File)`	Al fichero; lo vacía
`from(File)`	Lee del fichero
`appendTo(File)`	Añade, sin vaciar

Dos procesos en paralelo, cada uno a su fichero.




```
ProcessBuilder pb1 = new ProcessBuilder("sh", "-c", "ls");
pb1.redirectOutput(new File("output_ls.txt"));
Process process1 = pb1.start();   // idem con uname -a

int exitCode1 = process1.waitFor();
int exitCode2 = process2.waitFor();
```

Arrancan sin esperarse y trabajan a la vez; **waitFor()** los sincroniza y recoge sus códigos.

Lo esencial, en cinco líneas.

01	**Proceso**	Un programa en ejecución. Consume memoria, CPU y dispositivos de E/S.
02	**Planificación**	El SO reparte recursos buscando rendimiento, equidad y menores esperas.
03	**Concurrencia**	Mejora el rendimiento, pero exige comunicación y sincronización.
04	**Process API**	Se configura, se lanza con start() y se controla con waitFor() y exitValue().
05	**Flujos**	getInputStream, getOutputStream y las redirecciones gestionan la E/S.